



希望阻止拷贝的类应该使用 `=delete` 来定义它们自己的拷贝构造函数和拷贝赋值运算符，而不应该将它们声明为 `private` 的。

13.1.6 节练习

练习 13.18: 定义一个 `Employee` 类，它包含雇员的姓名和唯一的雇员证号。为这个类定义默认构造函数，以及接受一个表示雇员姓名的 `string` 的构造函数。每个构造函数应该通过递增一个 `static` 数据成员来生成一个唯一的证号。

练习 13.19: 你的 `Employee` 类需要定义它自己的拷贝控制成员吗？如果需要，为什么？如果不需要，为什么？实现你认为 `Employee` 需要的拷贝控制成员。

练习 13.20: 解释当我们拷贝、赋值或销毁 `TextQuery` 和 `QueryResult` 类（参见 12.3 节，第 430 页）对象时会发生什么。

练习 13.21: 你认为 `TextQuery` 和 `QueryResult` 类需要定义它们自己版本的拷贝控制成员吗？如果需要，为什么？如果不需要，为什么？实现你认为这两个类需要的拷贝控制操作。



13.2 拷贝控制和资源管理

通常，管理类外资源的类必须定义拷贝控制成员。如我们在 13.1.4 节（第 447 页）中所见，这类类需要通过析构函数来释放对象所分配的资源。一旦一个类需要析构函数，那么它几乎肯定也需要一个拷贝构造函数和一个拷贝赋值运算符。

为了定义这些成员，我们首先必须确定此类型对象的拷贝语义。一般来说，有两种选择：可以定义拷贝操作，使类的行为看起来像一个值或者像一个指针。

类的行为像一个值，意味着它应该也有自己的状态。当我们拷贝一个像值的对象时，副本和原对象是完全独立的。改变副本不会对原对象有任何影响，反之亦然。

行为像指针的类则共享状态。当我们拷贝一个这种类的对象时，副本和原对象使用相同的底层数据。改变副本也会改变原对象，反之亦然。

在我们使用过的标准库类中，标准库容器和 `string` 类的行为像一个值。而不出意外的，`shared_ptr` 类提供类似指针的行为，就像我们的 `StrBlob` 类（参见 12.1.1 节，第 405 页）一样，`IO` 类型和 `unique_ptr` 不允许拷贝或赋值，因此它们的行为既不像值也不像指针。

为了说明这两种方式，我们会为练习中的 `HasPtr` 类定义拷贝控制成员。首先，我们将令类的行为像一个值；然后重新实现类，使它的行为像一个指针。

我们的 `HasPtr` 类有两个成员，一个 `int` 和一个 `string` 指针。通常，类直接拷贝内置类型（不包括指针）成员；这些成员本身就是值，因此通常应该让它们的行为像值一样。我们如何拷贝指针成员决定了像 `HasPtr` 这样的类是具有类值行为还是类指针行为。

13.2 节练习

练习 13.22: 假定我们希望 `HasPtr` 的行为像一个值。即，对于对象所指向的 `string`

成员，每个对象都有一份自己的拷贝。我们将在下一节介绍拷贝控制成员的定义。但是，你已经学习了定义这些成员所需的所有知识。在继续学习下一节之前，为 `HasPtr` 编写拷贝构造函数和拷贝赋值运算符。

13.2.1 行为像值的类



为了提供类值的行为，对于类管理的资源，每个对象都应该拥有一份自己的拷贝。这意味着对于 `ps` 指向的 `string`，每个 `HasPtr` 对象都必须有自己的拷贝。为了实现类值行为，`HasPtr` 需要

- 定义一个拷贝构造函数，完成 `string` 的拷贝，而不是拷贝指针
- 定义一个析构函数来释放 `string`
- 定义一个拷贝赋值运算符来释放对象当前的 `string`，并从右侧运算对象拷贝 `string`

类值版本的 `HasPtr` 如下所示

```
class HasPtr {
public:
    HasPtr(const std::string &s = std::string()):
        ps(new std::string(s)), i(0) { }
    // 对 ps 指向的 string，每个 HasPtr 对象都有自己的拷贝
    HasPtr(const HasPtr &p):
        ps(new std::string(*p.ps)), i(p.i) { }
    HasPtr& operator=(const HasPtr &);
    ~HasPtr() { delete ps; }
private:
    std::string *ps;
    int i;
};
```

我们的类足够简单，在类内就已定义了除赋值运算符之外的所有成员函数。第一个构造函数接受一个（可选的）`string` 参数。这个构造函数动态分配它自己的 `string` 副本，并将指向 `string` 的指针保存在 `ps` 中。拷贝构造函数也分配它自己的 `string` 副本。析构函数对指针成员 `ps` 执行 `delete`，释放构造函数中分配的内存。

512

类值拷贝赋值运算符

赋值运算符通常组合了析构函数和构造函数的操作。类似析构函数，赋值操作会销毁左侧运算对象的资源。类似拷贝构造函数，赋值操作会从右侧运算对象拷贝数据。但是，非常重要的一点是，这些操作是以正确的顺序执行的，即使将一个对象赋予它自身，也保证正确。而且，如果可能，我们编写的赋值运算符还应该是异常安全的——当异常发生时能将左侧运算对象置于一个有意义的状态（参见 5.6.2 节，第 175 页）。

在本例中，通过先拷贝右侧运算对象，我们可以处理自赋值情况，并能保证在异常发生时代码也是安全的。在完成拷贝后，我们释放左侧运算对象的资源，并更新指针指向新分配的 `string`：

```
HasPtr& HasPtr::operator=(const HasPtr &rhs)
{
    auto newp = new string(*rhs.ps); // 拷贝底层 string
    delete ps; // 释放旧内存
```

```

    ps = newp;           // 从右侧运算对象拷贝数据到本对象
    i = rhs.i;
    return *this;       // 返回本对象
}

```

在这个赋值运算符中，非常清楚，我们首先进行了构造函数的工作：newp 的初始化器等价于 HasPtr 的拷贝构造函数中 ps 的初始化器。接下来与析构函数一样，我们 delete 当前 ps 指向的 string。然后就只剩下拷贝指向新分配的 string 的指针，以及从 rhs 拷贝 int 值到本对象了。

关键概念：赋值运算符

当你编写赋值运算符时，有两点需要记住：

- 如果将一个对象赋予它自身，赋值运算符必须能正确工作。
- 大多数赋值运算符组合了析构函数和拷贝构造函数的工作。

当你编写一个赋值运算符时，一个好的模式是先将右侧运算对象拷贝到一个局部临时对象中。当拷贝完成后，销毁左侧运算对象的现有成员就是安全的了。一旦左侧运算对象的资源被销毁，就只剩下将数据从临时对象拷贝到左侧运算对象的成员中了。

513

为了说明防范自赋值操作的重要性，考虑如果赋值运算符如下编写将会发生什么

```

// 这样编写赋值运算符是错误的！
HasPtr&
HasPtr::operator=(const HasPtr &rhs)
{
    delete ps; // 释放对象指向的 string
    // 如果 rhs 和 *this 是同一个对象，我们就将从已释放的内存中拷贝数据！
    ps = new string(*(rhs.ps));
    i = rhs.i;
    return *this;
}

```

如果 rhs 和本对象是同一个对象，delete ps 会释放 *this 和 rhs 指向的 string。接下来，当我们在 new 表达式中试图拷贝 *(rhs.ps) 时，就会访问一个指向无效内存的指针，其行为和结果是未定义的。



对于一个赋值运算符来说，正确工作是非常重要的，即使是将一个对象赋予它自身，也要能正确工作。一个好的方法是在销毁左侧运算对象资源之前拷贝右侧运算对象。

13.2.1 节练习

练习 13.23：比较上一节练习中你编写的拷贝控制成员和这一节中的代码。确定你理解了你的代码和我们的代码之间的差异（如果有的话）。

练习 13.24：如果本节中的 HasPtr 版本未定义析构函数，将会发生什么？如果未定义拷贝构造函数，将会发生什么？

练习 13.25：假定希望定义 StrBlob 的类值版本，而且希望继续使用 shared_ptr，

这样我们的 `StrBlobPtr` 类就仍能使用指向 `vector` 的 `weak_ptr` 了。你修改后的类将需要一个拷贝构造函数和一个拷贝赋值运算符，但不需要析构函数。解释拷贝构造函数和拷贝赋值运算符必须要做什么。解释为什么不需要析构函数。

练习 13.26: 对上一题中描述的 `StrBlob` 类，编写你自己的版本。

13.2.2 定义行为像指针的类



对于行为类似指针的类，我们需要为其定义拷贝构造函数和拷贝赋值运算符，来拷贝指针成员本身而不是它指向的 `string`。我们的类仍然需要自己的析构函数来释放接受 `string` 参数的构造函数分配的内存（参见 13.1.4 节，第 447 页）。但是，在本例中，析构函数不能单方面地释放关联的 `string`。只有当最后一个指向 `string` 的 `HasPtr` 销毁时，它才可以释放 `string`。

令一个类展现类似指针的行为的最好方法是使用 `shared_ptr` 来管理类中的资源。拷贝（或赋值）一个 `shared_ptr` 会拷贝（赋值）`shared_ptr` 所指向的指针。`shared_ptr` 类自己记录有多少用户共享它所指向的对象。当没有用户使用对象时，`shared_ptr` 类负责释放资源。

◀ 514

但是，有时我们希望直接管理资源。在这种情况下，使用引用计数（reference count）（参见 12.1.1 节，第 402 页）就很有用了。为了说明引用计数如何工作，我们将重新定义 `HasPtr`，令其行为像指针一样，但我们不使用 `shared_ptr`，而是设计自己的引用计数。

引用计数

引用计数的工作方式如下：

- 除了初始化对象外，每个构造函数（拷贝构造函数除外）还要创建一个引用计数，用来记录有多少对象与正在创建的对象共享状态。当我们创建一个对象时，只有一个对象共享状态，因此将计数器初始化为 1。
- 拷贝构造函数不分配新的计数器，而是拷贝给定对象的数据成员，包括计数器。拷贝构造函数递增共享的计数器，指出给定对象的状态又被一个新用户所共享。
- 析构函数递减计数器，指出共享状态的用户少了一个。如果计数器变为 0，则析构函数释放状态。
- 拷贝赋值运算符递增右侧运算对象的计数器，递减左侧运算对象的计数器。如果左侧运算对象的计数器变为 0，意味着它的共享状态没有用户了，拷贝赋值运算符就必须销毁状态。

唯一的难题是确定在哪里存放引用计数。计数器不能直接作为 `HasPtr` 对象的成员。下面的例子说明了原因：

```
HasPtr p1("Hiya!");  
HasPtr p2(p1);    // p1 和 p2 指向相同的 string  
HasPtr p3(p1);    // p1、p2 和 p3 都指向相同的 string
```

如果引用计数保存在每个对象中，当创建 `p3` 时我们应该如何正确更新它呢？可以递增 `p1` 中的计数器并将其拷贝到 `p3` 中，但如何更新 `p2` 中的计数器呢？

解决此问题的一种方法是将计数器保存在动态内存中。当创建一个对象时，我们也分配一个新的计数器。当拷贝或赋值对象时，我们拷贝指向计数器的指针。使用这种方法，副本和原对象都会指向相同的计数器。

定义一个使用引用计数的类

通过使用引用计数，我们就可以编写类指针的 HasPtr 版本了：

515

```
class HasPtr {
public:
    // 构造函数分配新的 string 和新的计数器，将计数器置为 1
    HasPtr(const std::string &s = std::string()):
        ps(new std::string(s)), i(0), use(new std::size_t(1)) {}
    // 拷贝构造函数拷贝所有三个数据成员，并递增计数器
    HasPtr(const HasPtr &p):
        ps(p.ps), i(p.i), use(p.use) { ++*use; }
    HasPtr& operator=(const HasPtr&);
    ~HasPtr();

private:
    std::string *ps;
    int i;
    std::size_t *use; // 用来记录有多少个对象共享*ps 的成员
};
```

在此，我们添加了一个名为 use 的数据成员，它记录有多少对象共享相同的 string。接受 string 参数的构造函数分配新的计数器，并将其初始化为 1，指出当前有一个用户使用本对象的 string 成员。

类指针的拷贝成员“篡改”引用计数

当拷贝或赋值一个 HasPtr 对象时，我们希望副本和原对象都指向相同的 string。即，当拷贝一个 HasPtr 时，我们将拷贝 ps 本身，而不是 ps 指向的 string。当我们进行拷贝时，还会递增该 string 关联的计数器。

（我们在类内定义的）拷贝构造函数拷贝给定 HasPtr 的所有三个数据成员。这个构造函数还递增 use 成员，指出 ps 和 p.ps 指向的 string 又有了一个新的用户。

析构函数不能无条件地 delete ps——可能还有其他对象指向这块内存。析构函数应该递减引用计数，指出共享 string 的对象少了一个。如果计数器变为 0，则析构函数释放 ps 和 use 指向的内存：

```
HasPtr::~~HasPtr()
{
    if (--*use == 0) { // 如果引用计数变为 0
        delete ps;    // 释放 string 内存
        delete use;   // 释放计数器内存
    }
}
```

拷贝赋值运算符与往常一样执行类似拷贝构造函数和析构函数的工作。即，它必须递增右侧运算对象的引用计数（即，拷贝构造函数的工作），并递减左侧运算对象的引用计数，在必要时释放使用的内存（即，析构函数的工作）。

而且与往常一样，赋值运算符必须处理自赋值。我们通过先递增 rhs 中的计数然后再递减左侧运算对象中的计数来实现这一点。通过这种方法，当两个对象相同时，在我们检查 ps（及 use）是否应该释放之前，计数器就已经被递增过了：

516

```
HasPtr& HasPtr::operator=(const HasPtr &rhs)
{
```

```

++*rhs.use; // 递增右侧运算对象的引用计数
if (--*use == 0) { // 然后递减本对象的引用计数
    delete ps; // 如果没有其他用户
    delete use; // 释放本对象分配的成员
}
ps = rhs.ps; // 将数据从 rhs 拷贝到本对象
i = rhs.i;
use = rhs.use;
return *this; // 返回本对象
}

```

13.2.2 节练习

练习 13.27: 定义你自己的使用引用计数版本的 HasPtr。

练习 13.28: 给定下面的类，为其实现一个默认构造函数和必要的拷贝控制成员。

<pre> (a) class TreeNode { private: std::string value; int count; TreeNode *left; TreeNode *right; }; </pre>	<pre> (b) class BinStrTree { private: TreeNode *root; }; </pre>
--	---

13.3 交换操作

除了定义拷贝控制成员，管理资源的类通常还定义一个名为 swap 的函数（参见 9.2.5 节，第 303 页）。对于那些与重排元素顺序的算法（参见 10.2.3 节，第 342 页）一起使用的类，定义 swap 是非常重要的。这类算法在需要交换两个元素时会调用 swap。

如果一个类定义了自己的 swap，那么算法将使用类自定义版本。否则，算法将使用标准库定义的 swap。虽然与往常一样我们不知道 swap 是如何实现的，但理论上很容易理解，为了交换两个对象我们需要进行一次拷贝和两次赋值。例如，交换两个类值 HasPtr 对象（参见 13.2.1 节，第 453 页）的代码可能像下面这样：

```

HasPtr temp = v1; // 创建 v1 的值的一个临时副本
v1 = v2; // 将 v2 的值赋予 v1
v2 = temp; // 将保存的 v1 的值赋予 v2

```

这段代码将原来 v1 中的 string 拷贝了两次——第一次是 HasPtr 的拷贝构造函数将 v1 拷贝给 temp，第二次是赋值运算符将 temp 赋予 v2。将 v2 赋予 v1 的语句还拷贝了原来 v2 中的 string。如我们所见，拷贝一个类值的 HasPtr 会分配一个新 string 并将其拷贝到 HasPtr 指向的位置。

◀ 517

理论上，这些内存分配都是不必要的。我们更希望 swap 交换指针，而不是分配 string 的新副本。即，我们希望这样交换两个 HasPtr：

```

string *temp = v1.ps; // 为 v1.ps 中的指针创建一个副本
v1.ps = v2.ps; // 将 v2.ps 中的指针赋予 v1.ps
v2.ps = temp; // 将保存的 v1.ps 中原来的指针赋予 v2.ps

```